



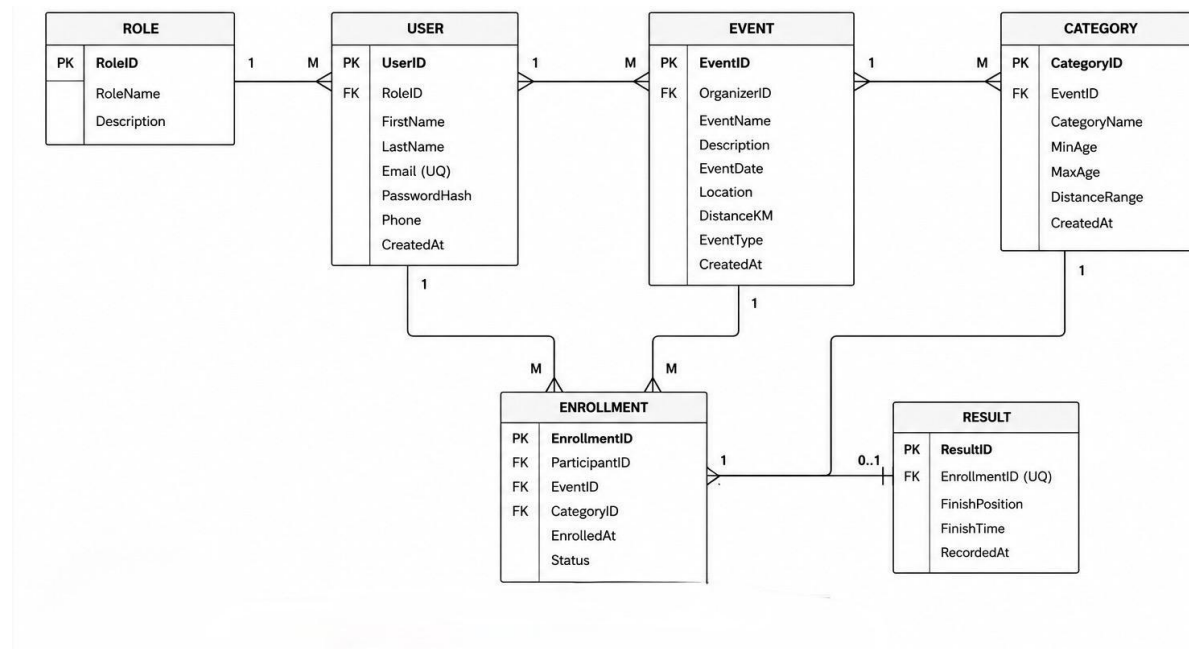
PROG212

Nothando Mokoka ST10477916



Race Day System Project

SECTION A: An Entity Relationship Diagram (ERD)



Entities and Attributes Summary

Role

- RoleID (PK) - unique role identifier
- Description - role description (Organizer or Participant)

User

- UserID (PK) - unique user identifier
- RoleID - user's role
- FirstName - user's first name
- LastName - user's surname
- Email - unique login or public email
- PasswordHash - stored password hash
- PhoneNumber - public contact number
- CreatedDate - account creation date

Event

- EventID (PK) - unique event identifier
- OrganizerID - organizer responsible for event
- EventName - event name
- EventDescription - event description
- EventDate - date of the event

- Location - event location
- DistanceKM - event distance
- EventType - running, walking or cycling
- CreatedAt – place created

Category

- CategoryID (PK) - unique category identifier
- EventID - event to which category belongs
- CategoryName - category name
- MinAge - minimum participant age, where applicable
- MaxAge - maximum participant age, where applicable
- DistanceRange - category distance, where applicable

Enrollment

- EnrollmentID (PK) - unique enrolment identifier
- ParticipantID(FK)- participant entering the event
- EventID (FK) - event entered
- CategoryID (FK) - selected category
- EnrollmentAt - place of enrolment
- Status - Status of enrolment

Result

- ResultID (PK) — unique result identifier
- EnrollmentID (FK → Enrollment.EnrollmentID) — enrolment receiving the result
- FinishPosition — participant's finishing position
- Finish Time — participant's finishing time
- RecordedAt – Place of record

Relationship Summary

Relationship	Cardinality	Foreign Key
Role & User	1 : M	User.RoleID
User(Organizer) & Event	1 : M	Event.OrganizerID
Event & Category	1 : M	Category.EventID
User(Participant) & Enrollment	1 : M	Enrollment.ParticipantID
Event & Enrollment	1 : M	Enrollment.EventID
Category & Enrollment	1 : M	Enrollment.CategoryID

Enrolment & Result	1 : 0..1	Result.EnrollmentID
--------------------	----------	---------------------

SECTION B: API Endpoint Plan

The endpoint plan covers authentication, user profiles, events, categories, event enrolment and results. Cases that may fail are included in the expected response column(where necessary).

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Register a new user account.	Public	{ "firstName": "...", "lastName": "...", "email": "...", "password": "...", "phoneNumber": "...", "roleId": 2 }	201 Created; 400 Bad Request; 409 Conflict if email exists.
POST	/api/auth/login	Authenticate user and return access token.	Public	{ "email": "...", "password": "... " }	200 OK with token/user/role; 401 Unauthorized.
GET	/api/users/me	View own profile.	Organizer / Participant	None	200 OK; 401 Unauthorized.
PUT	/api/users/me	Update own profile.	Organizer / Participant	{ "firstName": "...", "lastName": "...", "email": "...", "phoneNumber": "... " }	200 OK; 400 Bad Request; 401 Unauthorized; 409 Conflict.
GET	/api/events	View available events.	Organizer / Participant	None	200 OK with event list; 500 Internal Server Error.
GET	/api/events/{eventId}	View one event.	Organizer / Participant	None	200 OK; 404 Not Found.
POST	/api/events	Create an event.	Organizer	{ "name": "...", "description": "...", "eventDate": "YYYY-MM-DD", "location": "...", "distance": 10.0, "eventType": "Running" }	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden.
PUT	/api/events/{eventId}	Update an event managed by organizer.	Organizer	{ "name": "...", "description": "...", "eventDate": "YYYY-MM-DD", "location": "...", "distance": 10.0, "eventType": "Running" }	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found.
DELETE	/api/events/{eventId}	Delete an event managed by organizer.	Organizer	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found.
GET	/api/events/{eventId}/categories	View event categories.	Organizer / Participant	None	200 OK; 404 Not Found.
POST	/api/events/{eventId}/categories	Create an age/distance category.	Organizer	{ "categoryName": "Open 18-39", "minimumAge": 18, "maximumAge": 39, "distance": 10.00 }	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found.
PUT	/api/categories/{categoryId}	Update a category.	Organizer	{ "categoryName": "...", "minimumAge": 18, "maximumAge": 39, "distance": 10.00 }	200 OK; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found.
DELETE	/api/categories/{categoryId}	Delete a category.	Organizer	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found; 409 Conflict.
POST	/api/events/{eventId}/enrollments	Enter event and select category.	Participant	{ "categoryId": 1 }	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found; 409 Conflict.
GET	/api/users/me/enrollments	View own enrolments.	Participant	None	200 OK; 401 Unauthorized; 403 Forbidden.
GET	/api/events/{eventId}/enrollments	View all enrolments for own event.	Organizer	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found.
GET	/api/enrollments/{enrollmentId}	View one own enrolment.	Participant	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found.
POST	/api/enrollments/{enrollmentId}/result	Capture finish position and time.	Organizer	{ "finishPosition": 1, "finishTime": "01:05:30" }	201 Created; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found; 409 Conflict.
GET	/api/users/me/results	View own results.	Participant	None	200 OK; 401 Unauthorized; 403 Forbidden.
GET	/api/events/{eventId}/results	View results for own event.	Organizer	None	200 OK; 401 Unauthorized; 403 Forbidden; 404 Not Found.

SECTION C: Database Script

1. Database setup

- Checks if RaceDay.sql already exists.
- If it exists it deletes it
- Then it creates a fresh RaceDay.sql database and selects it for use.

2. Creates 6 tables

- Role : stores user roles such as Organizer and Participant
- User: stores user details and links each user to a role.
- Event: stores race/walk/cycling event information.
- Category: stores categories for each event including age and distance
- .Enrollment : records which participants register for which events/categories.
- Result: stores participants' finishing positions and times.

3. Relationships

- Foreign keys connect the tables.

4. Validation

- PRIMARY KEY uniquely identifies records.
- FOREIGN KEY maintains relationships.
- UNIQUE prevents duplicates such as two users having the same email.
- CHECK constraints prevent invalid data such as a negative distance.

5. Sample data

- Adds 2 roles, 4 users, 3 events, event categories, enrollments and sample race results.

6. Verification

- The final SELECT * statements display all the tables so you can check that the database was created and populated correctly.

References

- Troelsen, A. & Japikse, P. (2021). Pro C# 9 with .NET 5: Foundational Principles and Practices in Programming. 10th ed. Apress.
- Functional requirements: Race Day POE Part 1 and Part 2.